

Consigna general

Todos los ejercicios deberán desarrollarse en **Java** utilizando **OpenCode**.

Antes de solicitar la implementación, deberán analizar el problema y redactar un **prompt** que describa claramente cómo desean resolverlo mediante recursividad (caso base, caso recursivo y cómo se reduce el problema).

Al comenzar cada archivo `.java`, deberán incluir como comentario el **prompt inicial** utilizado para generar la solución.

Si durante el desarrollo realizaron modificaciones al prompt para mejorar el resultado, deberán agregar al final del archivo un comentario indicando **qué cambios hicieron y por qué**.

Se evaluará tanto el correcto funcionamiento del programa como la calidad del razonamiento reflejado en el prompt.

Ejercicio 1: Factorial

Escribir un prompt para OpenCode que permita implementar en Java una función recursiva para calcular el factorial de un número.

El prompt debe explicar:

- por qué el factorial puede resolverse de forma recursiva;
 - cuál es el caso base;
 - qué ocurre cuando `n` vale 0 o 1;
 - cómo se reduce el problema;
 - qué valores de prueba deberían usarse.
-

Ejercicio 2: Suma de los primeros N números

Escribir un prompt para OpenCode para implementar una función recursiva que sume los números desde `n` hasta 1.

El prompt debe explicar:

- cuál es el caso más simple;

- por qué `suma(n)` puede pensarse como $n + \text{suma}(n - 1)$;
 - qué debe pasar si `n` es 0;
 - cómo validar que la función funciona correctamente.
-

Ejercicio 3: Multiplicación mediante sumas

Escribir un prompt para OpenCode para implementar en Java una multiplicación recursiva sin usar el operador `*`.

El prompt debe explicar:

- cómo representar una multiplicación como sumas repetidas;
 - cuál de los dos parámetros se reduce;
 - por qué el caso base debería ser cuando uno de los factores vale 0;
 - qué debe devolver la función;
 - qué pruebas deberían hacerse.
-

Ejercicio 4: Potencia

Escribir un prompt para OpenCode para implementar una función recursiva que calcule una potencia sin usar `Math.pow`.

El prompt debe explicar:

- por qué una potencia puede verse como multiplicaciones sucesivas;
 - por qué el caso base es exponente 0;
 - cómo se reduce el exponente;
 - qué debe devolver la función en cada caso;
 - qué casos de prueba usar.
-

Ejercicio 5: Conteo regresivo

Escribir un prompt para OpenCode para implementar una función recursiva que imprima un conteo desde `n` hasta 0.

El prompt debe explicar:

- qué significa que la función no necesariamente devuelva un valor;
 - cuál es el caso base;
 - cómo se acerca al caso base;
 - qué pasaría si en lugar de $n - 1$ se usara $n + 1$;
 - cómo probar la ejecución.
-

Ejercicio 6: Contar dígitos

Escribir un prompt para OpenCode para implementar una función recursiva que cuente los dígitos de un número entero positivo.

El prompt debe explicar:

- cómo se puede reducir el número usando división entera;
 - cuál es el caso base;
 - por qué un número menor que 10 tiene un solo dígito;
 - qué debe devolver cada llamada;
 - qué valores de prueba usar.
-

Ejercicio 7: Sumar dígitos

Escribir un prompt para OpenCode para implementar una función recursiva que sume los dígitos de un número entero positivo.

El prompt debe explicar:

- cómo obtener el último dígito usando módulo;
 - cómo reducir el número usando división entera;
 - cuál es el caso base;
 - cómo se combinan los resultados;
 - qué pruebas validar.
-

Ejercicio 8: Invertir una palabra

Escribir un prompt para OpenCode para implementar una función recursiva que invierta un `String`.

El prompt debe explicar:

- cuál es el caso base para una cadena vacía o de un solo carácter;
 - cómo separar el primer carácter del resto;
 - cómo se reduce la cadena en cada llamada;
 - cómo se reconstruye el resultado al volver de la recursión;
 - qué palabras usar como prueba.
-

Ejercicio 9: Palíndromo

Escribir un prompt para OpenCode para implementar una función recursiva que determine si una palabra es palíndromo.

El prompt debe explicar:

- por qué hay que comparar el primer y último carácter;
 - qué pasa si son distintos;
 - cómo se reduce el problema eliminando extremos;
 - cuál es el caso base;
 - qué pruebas deberían hacerse.
-

Ejercicio 10: Buscar un elemento en un arreglo

Escribir un prompt para OpenCode para implementar una función recursiva que busque un número dentro de un arreglo, sin usar ciclos.

El prompt debe explicar:

- desde qué posición empieza la búsqueda;
 - cómo avanzar en el arreglo sin usar `for` ni `while`;
 - cuál es el caso base cuando encuentra el valor;
 - cuál es el caso base cuando llega al final;
 - qué debe devolver la función.
-

Ejemplo de formato esperado

Para cada ejercicio, el alumno debería entregar algo parecido a esto:

Quiero implementar en Java una función recursiva que resuelva [problema].

Este problema puede resolverse recursivamente porque [explicación].

El caso base es [condición], porque [motivo].

El caso recursivo es [llamada recursiva], porque en cada llamada [explicación de cómo se reduce el problema].

La función debe devolver [resultado esperado].

Quiero que el código tenga comentarios explicando el caso base, el caso recursivo y la pila de llamadas.

También quiero probarlo con estos casos:

- caso 1
- caso 2
- caso 3